Expt 8	Multirate Signal Processing
Date:	8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB

Code:

```
clc;
clear;
close all;

%% Original Sequence
n = 0:40;
x = sin(0.3*pi*n);

%% Decimation by 2
y = x(1:2:end);          % Keep every second sample
nd = 0:length(y)-1;

%% Interpolation by 2 (Zero Insertion)
v = zeros(1,2*length(x)); % Allocate memory
v(1:2:end) = x;          % Insert original samples at odd indices
ni = 0:length(v)-1;

%% Figure 1: Original Sequence
figure;
stem(n,x,'filled','LineWidth',1.2);
grid on;
title('Original Sequence x[n]');
xlabel('Sample Index (n)');
ylabel('Amplitude');

%% Figure 2: Decimated Sequence
figure;
stem(nd,y,'filled','LineWidth',1.2);
grid on;
title('Decimated Sequence (Factor = 2)');
xlabel('Sample Index (n)');
ylabel('Amplitude');

%% Figure 3: Interpolated Sequence
figure;
stem(ni,v,'filled','LineWidth',1.2);
grid on;
title('Interpolated Sequence (Factor = 2)');
xlabel('Sample Index (n)');
ylabel('Amplitude');

%% Figure 4: Comparison
figure;
```

```
stem(n,x,'b','filled','LineWidth',1.2);
hold on;
stem(0:2:length(v)-2,v(1:2:end),'r','filled','LineWidth',1.2);
grid on;

legend('Original Sequence','Interpolated Samples');
title('Comparison of Original and Interpolated Sequences');
xlabel('Sample Index (n)');
ylabel('Amplitude');
```

Output:

Fig 1:

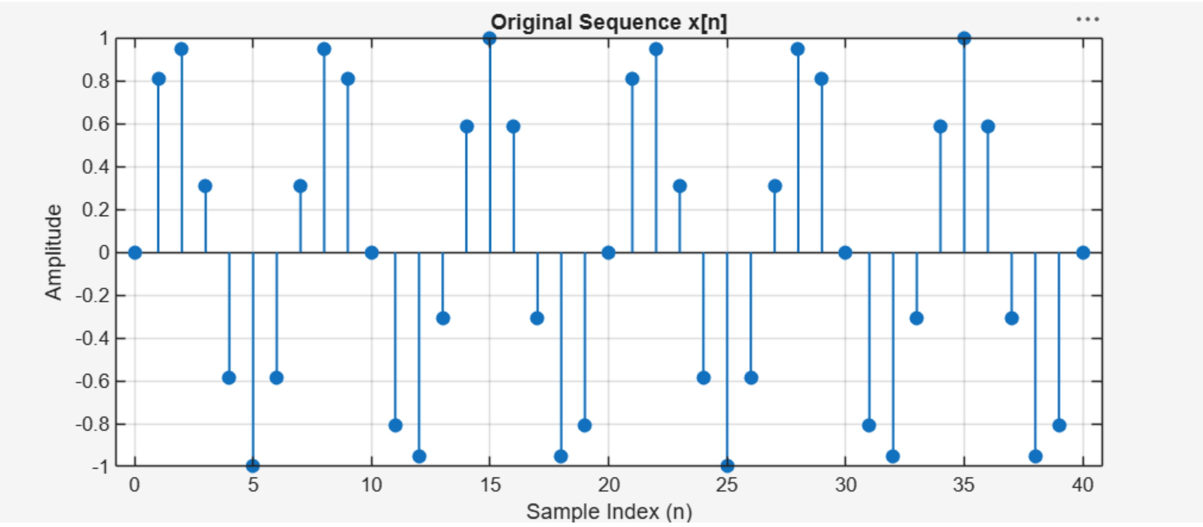


Fig 2:

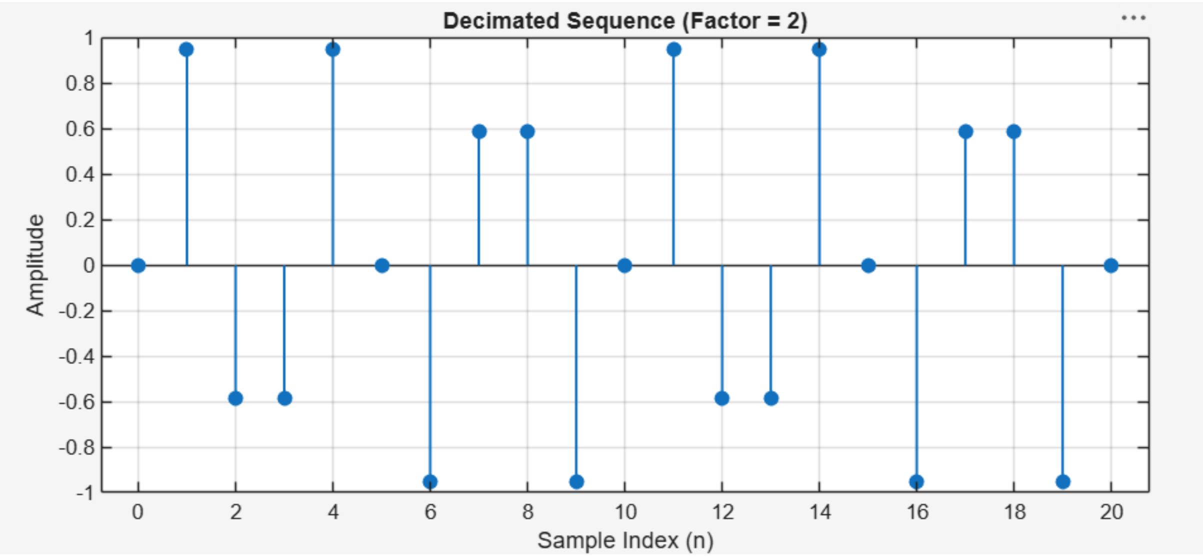


Fig 3:

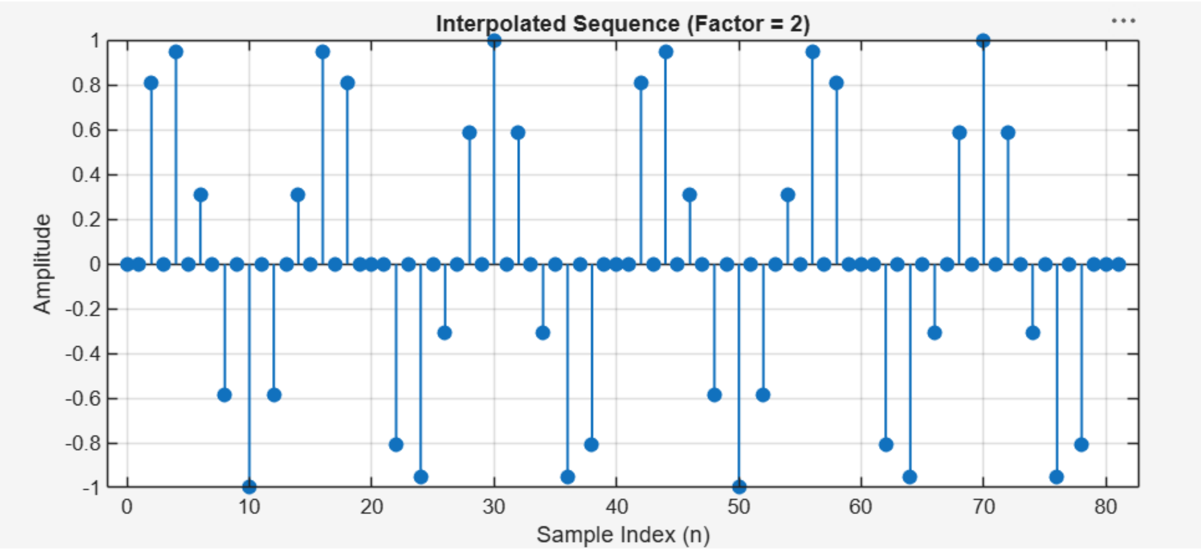
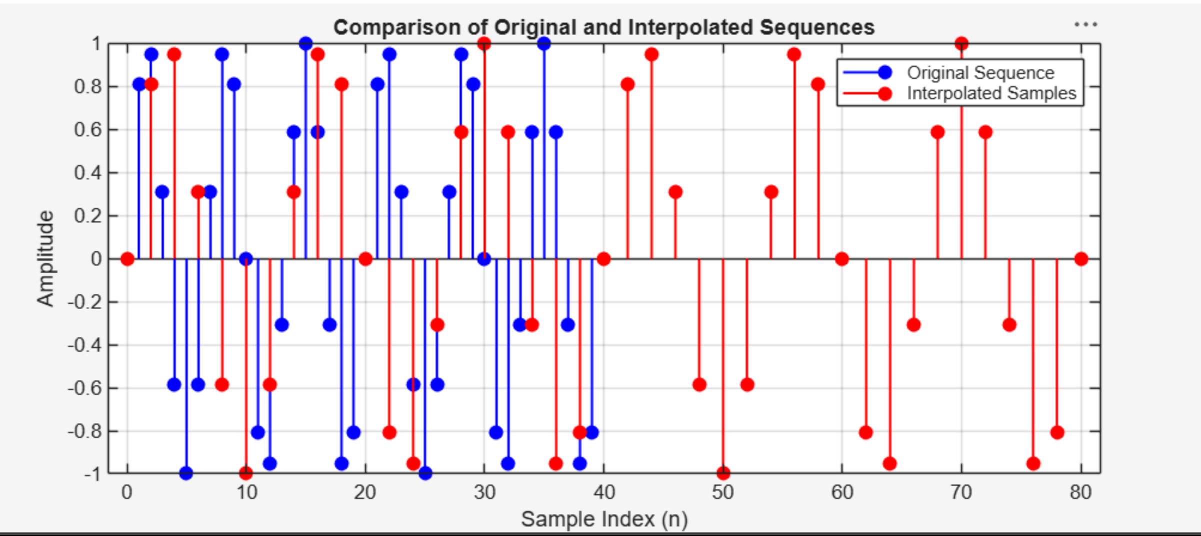

Interpolated Sequence (Factor = 2)
Amplitude
Sample Index (n)
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
10
20
30
40
50
60
70
80

Fig 4:


Comparison of Original and Interpolated Sequences
Original Sequence
Interpolated Samples
Amplitude
Sample Index (n)
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
10
20
30
40
50
60
70
80

Expt 8	Multirate Signal Processing
Date:	8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB

Code:

```
clc;
clear;
close all;

%% Input Signal
Fs = 1000;                  % Sampling Frequency (Hz)
t = 0:1/Fs:1;               % Time Vector

% Test Signal
x = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t);

%% FIR Low-Pass Filter Design
N = 20;                     % Filter Order
Fc = 0.4;                   % Normalized Cutoff Frequency
h = fir1(N, Fc, hamming(N+1));

%% Decimation Factor
M = 2;

% Polyphase Decimation
y_dec = upfirdn(x, h, 1, M);

%% Interpolation Factor
L = 2;

% Polyphase Interpolation
y_int = upfirdn(y_dec, h, L, 1);

%% Adjust Length for Comparison
if length(y_int) > length(x)
    y_int = y_int(1:length(x));
elseif length(y_int) < length(x)
    x = x(1:length(y_int));
end

%% Time Axes
t1 = (0:length(x)-1)/Fs;
t2 = (0:length(y_dec)-1)/(Fs/M);
t3 = (0:length(y_int)-1)/Fs;

%% Plot Results
```

```
figure('Name','Polyphase Decimation and Interpolation','NumberTitle','off');

% Original Signal
subplot(4,1,1);
plot(t1,x,'b','LineWidth',1.5);
title('Original Input Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

% Decimated Signal
subplot(4,1,2);
stem(t2,y_dec,'r','filled');
title('Decimated Signal (M = 2)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

% Interpolated Signal
subplot(4,1,3);
plot(t3,y_int,'k','LineWidth',1.5);
title('Interpolated/Reconstructed Signal (L = 2)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

% Comparison
subplot(4,1,4);
plot(t1,x,'b','LineWidth',1.5);
hold on;
plot(t3,y_int,'r--','LineWidth',1.5);
title('Comparison of Original and Reconstructed Signals');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal','Reconstructed Signal');
grid on;

%% Display Information
disp('--------------------------------------');
disp('POLYPHASE DECIMATION & INTERPOLATION');
disp('--------------------------------------');

fprintf('Original Signal Length      = %d\n', length(x));
fprintf('Decimated Signal Length     = %d\n', length(y_dec));
fprintf('Interpolated Signal Length  = %d\n', length(y_int));

%% Separate Comparison Figure
figure;
plot(t1,x,'b','LineWidth',2);
hold on;
```

```
plot(t3,y_int,'r--','LineWidth',2);
title('Original vs Reconstructed Signal');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal','Reconstructed Signal');
grid on;
```

Output:

Fig 1:

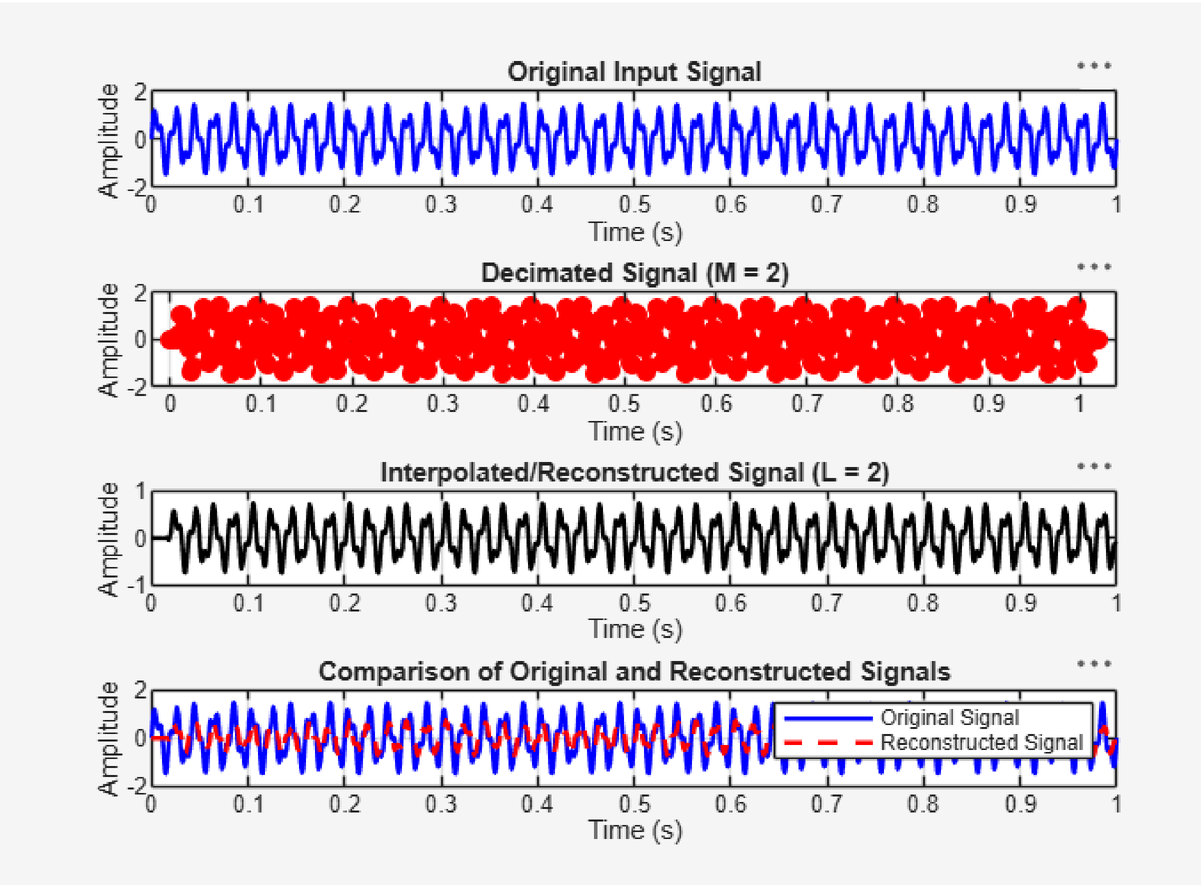
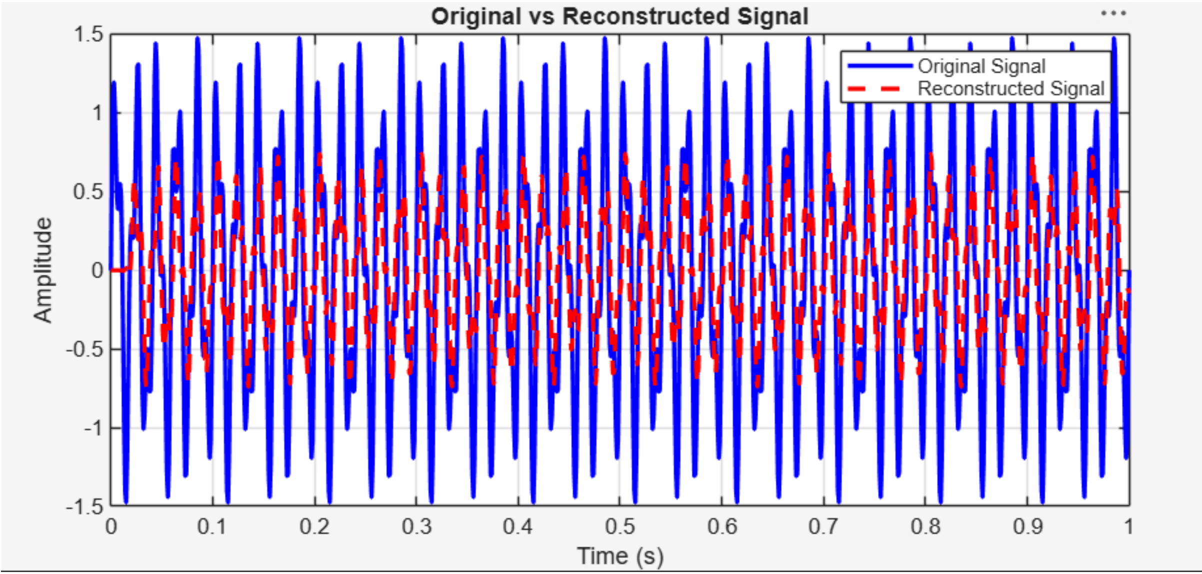


Fig 2:


Original vs Reconstructed Signal
Original Signal
Reconstructed Signal
Amplitude
Time (s)
1.5
1
0.5
0
-0.5
-1
-1.5
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1